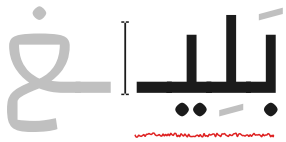
Cairo University
Faculty of Engineering
Department of Computer Engineering



Baligh

Arabic Writing Assistant

A Graduation Project Report Submitted

to

Faculty of Engineering, Cairo University

in Partial Fulfillment of the requirements of the degree

of

Bachelor of Science in Computer Engineering.

Presented by

Ahmed Hamed Gaber Hamed

Supervised by

Dr. Ayman AboElhassan

July, 2026

Abstract

Arabic digital writing still suffers from a shortage of integrated tools that can correct Modern Standard Arabic text accurately, respond in real time, and explain their feedback in a way that helps users learn. Existing tools often focus on surface spelling fixes or black-box suggestions without exposing the grammatical reasoning behind the correction. Baligh addresses this problem by providing an Arabic writing assistant that combines grammatical error detection, grammatical error correction, spelling-aware editing, next-word suggestion, and explanatory feedback in a single user-facing system. The project aims to support students, educators, and Arabic content writers by improving writing quality while also making the correction process more understandable and educational.

The implemented system follows a modular pipeline. A preprocessing layer performs normalization, word-boundary handling, segmentation, and morphological analysis and disambiguation. On top of that, Baligh integrates a grammatical error detection engine that fuses rule-based, lexicon-based, and machine-learning detectors, then passes its findings to correction components built around ontology-guided reasoning, dictionary lookup, and text-editing models. In parallel, a next-word suggestion module supports both next-word prediction and word auto-completion. The final system highlights issues, proposes corrections, returns ranked suggestions, and presents grammar-oriented explanations to the user in an interactive workflow.

The project outputs include the preprocessing pipeline, grammatical error detection and correction services, the next-word suggestion subsystem, and a working user application. Testing and validation relied on automated unit and integration tests together with evaluation on established Arabic linguistic resources, including QALB14, QALB15, and ZAEBUC. Overall, Baligh demonstrates that a modular and explainable Arabic writing assistant can combine linguistic processing and machine learning into a practical platform for improving Arabic writing quality.

Contents

List of Figures

List of Abbreviations

Abbreviation	Meaning
AI	Artificial Intelligence
API	Application Programming Interface
CRF	Conditional Random Field
GEC	Grammatical Error Correction
GED	Grammatical Error Detection
GUI	Graphical User Interface
MSA	Modern Standard Arabic
NLP	Natural Language Processing
NWP	Next-Word Prediction
POS	Part of Speech
QALB	Qatar Arabic Language Bank

Chapter 1: Introduction

1.1. Project Overview

Baligh is an Arabic writing assistant designed to support Modern Standard Arabic writing through explainable correction and real-time assistance. This chapter introduces the motivation behind the project, defines the problem it addresses, summarizes the main outcomes delivered by the project, and outlines the organization of the remainder of the report.

1.2. Individual Role Overview

My individual contribution was centered on the research and implementation of the Grammatical Error Correction (GEC) module and the design of the core Backend API architecture. I was responsible for bridging the gap between error detection and explainable correction by building two powerful subsystems: the Dictionary Engine and the Ontology Engine. This work involved deeply understanding Semantic Web technologies to model Arabic grammar, integrating the Arramooz lexicon for rapid morphological generation, and engineering the core asynchronous REST API that serves these linguistic analyses to the frontend client.

In addition to the core GEC and API work, I ensured that the various NLP components (including the GED detectors) were seamlessly orchestrated behind scalable, documented API contracts. These supporting contributions are presented in this report as delivery-enabling work around the main technical contributions.

1.3. Document Organization

This booklet presents my role, the technical preparation that informed my work (focusing on Ontology and Semantic Web methodologies), the GEC-focused contribution chapter detailing the Dictionary and Ontology modules, the Backend API architecture, and finally the main validation results, lessons learned, and future improvements.

Chapter 2: Personal Role and Responsibilities

2.1. Team Responsibilities

My responsibilities during the project combined linguistic research, knowledge engineering, and backend system architecture. On the research side, I studied ”Generation then Comparison” correction methodologies and Semantic Web tools to model Arabic syntax rules formally. On the implementation side, I owned the Dictionary Engine, the Ontology Engine, and the orchestration of the overarching FastAPI backend. I also participated in ensuring the system’s scalability by setting up the database layer (MongoDB) and drafting the data validation contracts (Pydantic schemas) used across the application.

2.2. Assigned Tasks

My assigned tasks can be summarized as follows:

- Conduct an extensive literature review on Arabic grammatical error correction, specifically focusing on explainable, ontology-driven generation methods.
- Design and implement the GEC Dictionary Engine, integrating the Arramooz lexical database to support Out-Of-Vocabulary (OOV) detection and morphological generation.
- Develop the GEC Ontology Engine, mapping CAMeL morphology features to Web Ontology Language (OWL) properties, and utilizing SPARQL queries to generate syntactically correct word pairs.
- Architect and implement the core asynchronous Backend API using FastAPI, Uvicorn, and Motor, enabling the frontend to reliably request drafts, analysis, suggestions, and corrections.
- Design the API orchestration layer to unify outputs from disparate GED and GEC subsystems into a cohesive, editor-friendly JSON response.

2.3. Timeline and Deliverables

Table 2.1: *Major Milestones in My Individual Contribution*

Phase	Main Focus	Deliverables
Research phase	Survey Arabic GEC literature, Semantic Web (OWL/RDF), and Dictionary design patterns	Research notes, literature synthesis, and API architecture direction
Dictionary phase	Build the morphological generator and OOV detector	SQLite Arramooz integration, Dictionary Engine pipelines
Ontology phase	Model Arabic syntax and implement ”Generation then Comparison”	OWL ontology mappings, SPARQL query generators, and rule explanations
Backend API phase	Architect the scalable REST interface and data contracts	FastAPI routers, Pydantic schemas, and MongoDB controllers
Integration phase	Connect the GED and GEC engines behind unified endpoints	Orchestrated API responses, latency optimization, and final integration checks

Chapter 3: Knowledge Acquisition and Technical Preparation

3.1. Investigated Papers

The literature review for my contribution focused heavily on Semantic Web technologies and their application to Arabic natural language processing. I deeply studied the "Generation then Comparison" methodology proposed by Moukrim et al., which demonstrated the viability of using the Web Ontology Language (OWL) to model the Ontology of Arabic Syntax (OAS) [2]. This paper was foundational for the GEC Ontology Engine, as it outlined how SPARQL queries could enforce morphological and syntactic agreements (such as Case, Gender, and Number) by navigating a structured graph of Arabic grammar axioms. I also researched lexicography integration strategies to understand how a large-scale dictionary like Arramooz could be utilized for Out-Of-Vocabulary (OOV) detection and morphological generation.

3.2. Self-Learning Materials

To implement the project effectively, I relied on a mix of coursework, research papers, library documentation, experimentation notebooks, and framework references. The main self-learning inputs were:

- **College Machine Learning course by Dr. Yahia Zakaria**: strengthened the theoretical background behind the statistical modeling side of the project.
- **College Natural Language course by Dr. Sandra Wahid**: reinforced the language-processing concepts most relevant to Arabic NLP work.
- **Arabic NLP pipeline study**: focused on the preprocessing and morphology-analysis workflow used in the project, especially segmentation, disambiguation, and morphological feature access.
- **GEC modeling study**: read papers and studied semantic web and ontology-based approaches to Arabic grammatical error correction.
- **Software and integration study**: included FastAPI for service integration, and Python testing workflows for unit and integration validation.

Chapter 4: Individual Contributions

4.1. Contribution 1: The GEC Dictionary and Ontology Engines

The main technical contribution presented in this booklet is the design and implementation of the Baligh Grammatical Error Correction (GEC) subsystem. The objective was to build a correction engine that goes beyond simple spelling suggestions to provide grammatically aware, structurally sound, and fully explainable Arabic sentence corrections. To achieve this, I implemented a dual-engine architecture: the Dictionary Engine for morphological generation and Out-Of-Vocabulary handling, and the Ontology Engine for structured grammatical reasoning.

4.1.1. Overall System Context

The implemented GEC service receives error spans identified by upstream Grammatical Error Detection (GED) modules. It runs generation pipelines on these localized spans to propose corrections that adhere to strict Semantic Web rules. The high-level system context of Baligh is shown in Figure 4.1, situating the Dictionary and Ontology Engines within the backend API layer.

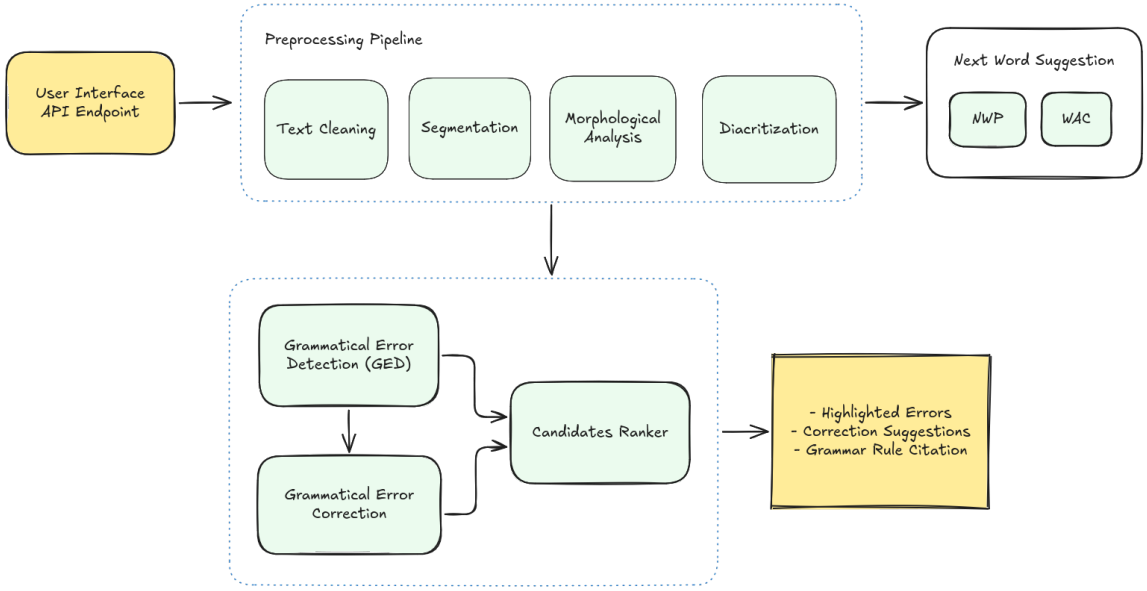


Figure 4.1: *High-level Baligh architecture showing the API Layer, Database Layer, and Core Modules.*

4.1.2. The Dictionary Engine

Functional Description

The Dictionary Module is responsible for robust spelling correction, Out-Of-Vocabulary (OOV) detection, and serving as a morphological generation backbone for the ontology. The Arabic language presents unique challenges in lexical validation due to its highly inflectional nature. To address this, the module utilizes the **Arramooz** dictionary, an open-source comprehensive Arabic lexicon organized in relational database tables. Arramooz encapsulates approximately 6,000 roots and covers stop words, nouns, and verbs.

The complete dictionary pipeline involves detecting OOV words by stripping clitics (prefixes/suffixes) to isolate the stem, querying the database for close string matches, and retrieving spelling candidates.

Modular Decomposition

The Dictionary Engine consists of the following submodules:

- **Spelling Checker:** Analyzes the incoming tokens and performs stem-based clitic analysis. Then it detects Out-Of-Vocabulary (OOV) words and generates a list of spelling candidates using edit distance metrics.
- **Arramooz Client:** A wrapper designed for efficient, low-latency querying of the SQLite-based Arramooz database.
- **Morphological Generator:** Acts upon requests from the Ontology Engine. When a specific inflection is required (e.g., converting a singular noun to a plural form in the accusative case), this generator queries Arramooz to construct the morphologically correct word form.
- **Alternative Ranker:** Ranks spelling candidates using Levenshtein distance (edit distance) combined with corpus-frequency metrics to present the most probable correction first.

Design Constraints

The primary constraint of the Dictionary Module is the reliance on raw string similarity (edit distance) for initial spelling suggestions. Edit distance lacks context-awareness. To counter this, the Dictionary Engine relies on downstream contextual rankers (like the

Machine Learning fusion layer) to make the final decision among its proposed alternatives.

Results

Figure 4.2: *Spelling Error (Original)*

Figure 4.3: *Dictionary Correction*

Figure 4.4: *Typographical Error (Original)*

Figure 4.5: *Dictionary Correction*

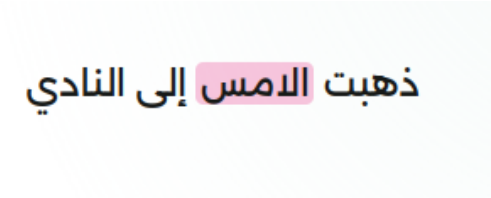


Figure 4.6: *Lexical Error (Original)*

Figure 4.7: *Dictionary Correction*

4.1.3. The Ontology Engine

Functional Description

An **Ontology** in computer science is a formal, explicit specification of a shared conceptualization. It defines a set of concepts and categories in a subject area, showing their properties and the relations between them. In the context of Baligh, the Ontology Module encodes Arabic syntax rules into a computational format, inspired by dependency grammar based on predicate logic.

This approach leverages **Semantic Web** technologies. The semantic web operates on layers, primarily the Resource Description Framework (RDF), which represents data as subject-predicate-object triples. For Arabic grammar, the **Ontology of Arabic Syntax (OAS)** is modeled using **OWL** (Web Ontology Language), an extension of RDF that allows for highly expressive logic and reasoning. In the OAS, concepts represent grammatical categories (e.g., Noun, Verb) while relationships represent syntactic links (e.g., Subject-Verb dependency) and functional properties (e.g., Case, Gender).

The methodology employed is “Generation then Comparison”:

1. **Categorization:** Words are assigned Lexical Features (meaning-heavy aspects like Transitivity) and Functional Features (structural aspects like Number, Gender, Case).
2. **Merger (Generation):** Using **SPARQL** (the query language for RDF), the system merges words into oriented grammatical pairs. By querying the OAS, the system generates all syntactically correct combinations of the sentence that abide by Arabic grammar axioms.
3. **Comparison:** The original incorrect sentence is compared against the generated valid sentences using string distance metrics, allowing the system to deduce the required correction.

Modular Decomposition

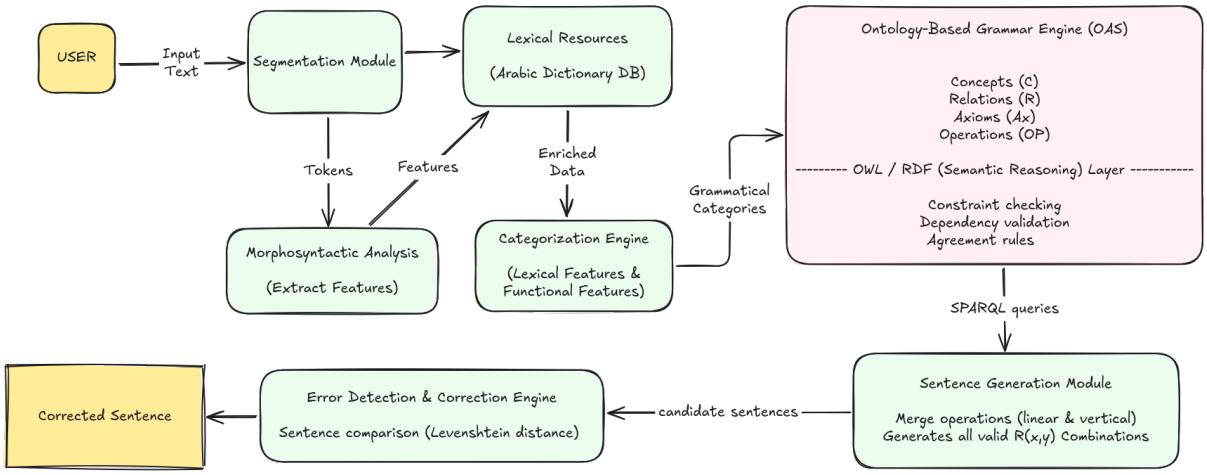


Figure 4.8: *Architecture of the GEC Module, showcasing the interaction between the Ontology Engine and the Dictionary Engine.*

The Ontology Engine orchestrates the generation logic via the following components:

- **Feature Mapper:** Translates CAMeL morphological outputs into the specific OWL terminology required by the OAS.
- **SPARQL Query Generator:** Constructs dynamic semantic queries. For example, it can query the ontology to enforce the axiom that a Verb's Subject must possess a Nominative case.
- **Candidate Generator:** Combines the theoretical rules from the ontology with the Morphological Generator from the Dictionary to produce the final corrected string phrase.
- **Explanation Generator:** Utilizes the specific violated ontology axiom to dynamically construct an educational, human-readable grammatical explanation for the user.

Design Constraints

Generating every possible syntactically correct permutation of a full sentence is computationally prohibitive, posing a significant latency risk for a real-time system. To circumvent this, the generation phase is aggressively localized. Instead of processing the entire sentence, the Ontology Engine restricts its generation only to the precise error spans previously identified by the Grammatical Error Detection (GED) module. Furthermore, bridging the terminology gap between the morphological analyzer's tags and

the OAS concepts required building a rigid, robust mapping dictionary to prevent query failures.

Results

Figure 4.9: *Grammatical Error (Original)*

Figure 4.10: *Ontology Correction*

Figure 4.11: *Syntactic Mismatch (Original)*

Figure 4.12: *Ontology Correction*

Figure 4.13: *Case/Gender Error (Original)*

Figure 4.14: *Ontology Correction*

4.2. Contribution 2: Core Backend API Architecture

While the GEC engines formed my primary research ownership area, I was also responsible for engineering the delivery mechanism that makes these linguistic models accessible: the Baligh Backend API.

4.2.1. FastAPI and Uvicorn Orchestration

The backend was built using FastAPI, chosen for its native asynchronous capabilities and automatic OpenAPI documentation generation. Hosted on Uvicorn (an ASGI web server), the backend is capable of handling multiple concurrent requests without blocking the event loop. This non-blocking architecture is critical when waiting for heavy Machine Learning inference or prolonged SPARQL queries. The API is divided into logical routers (`analysis.py`, `suggestions.py`, `corrections.py`, `tashkeel.py`) to maintain a clean separation of concerns.

4.2.2. Pydantic Data Contracts

To ensure robust communication between the React frontend and the Python backend, I heavily utilized Pydantic for data validation. Every request payload and response structure is defined by strict schema models. This prevented malformed frontend requests from crashing the NLP engines and guaranteed that the editor always received predictably formatted analysis results.

4.2.3. Asynchronous MongoDB Integration

For persistence (such as saving user drafts, tracking error frequencies, and managing document histories), I integrated MongoDB using the Motor asynchronous driver. This allowed the API to perform non-blocking database I/O, keeping the endpoints highly responsive even under load.

4.3. Testing and Validation

My contributions were validated through extensive unit and integration tests. The Dictionary Engine was tested against known OOV datasets to verify the accuracy of the Levenshtein distance rankings. The Ontology Engine's SPARQL queries were validated using a gold-standard set of manually annotated grammatical violations. Finally, the Backend API was stress-tested using `pytest-asyncio` to ensure that the routers gracefully handled high concurrency and malformed payloads without dropping connections.

Chapter 5: Challenges, Lessons Learned, and Future Work

5.1. Faced Challenges

The most significant challenge was bridging the terminology gap between the morphological outputs of CAMeL Tools and the Web Ontology Language (OWL) properties required by the Ontology of Arabic Syntax (OAS). Because the ontology expects highly formalized axioms, creating a robust mapping dictionary was crucial to avoid SPARQL query failures. A secondary challenge was managing latency; generating all correct permutations of a sentence using SPARQL is computationally expensive, which required aggressive localization of the generation phase to only operate on specific GED error spans. Finally, designing the unified Backend API required careful orchestration to handle these heavy synchronous NLP tasks without blocking the asynchronous event loop.

5.2. Lessons Learned

- **Knowledge Representation**: Semantic Web technologies (RDF/OWL) provide a powerful, explainable alternative to black-box machine learning for rigid linguistic rule systems.
- **Data Contracts**: Enforcing strict Pydantic schemas between the frontend and backend is essential for preventing silent failures and guaranteeing predictable user experiences.

5.3. Future Enhancements

Future iterations of the GEC modules could expand the Ontology of Arabic Syntax (OAS) to cover more complex rhetorical structures and semantic dependencies beyond basic morpho-syntactic agreement. Additionally, the SPARQL Query Generator could be optimized further through caching mechanisms (like Redis) for frequently corrected phrases to reduce latency.

References

[1] B. Alhafni, N. Inoue, and N. Habash, “Advancements in Arabic Grammatical Error Correction: The Promise of Large Language Models and a Two-Stage System,” in *Proceedings of ArabicNLP*, 2023.

[2] M. Moukrim, A. Namir, and M. Boulaknadel, “An Innovative Approach for Autocorrecting Arabic Syntactic Errors Based on Ontology,” *International Journal of Advanced Computer Science and Applications*, 2021.

[3] W. Zaghouani, N. Habash, O. Obeid, H. Bouamor, and S. Khalifa, “SAHSOH@QALB-2015 Shared Task: A Hybrid Arabic Spelling and Grammar Correction System,” in *Proceedings of the Second Workshop on Arabic Natural Language Processing*, 2015.

[4] R. Belkebir and N. Habash, “Automatic Error Type Annotation for Arabic,” in *Proceedings of the 6th Arabic Natural Language Processing Workshop*, 2021.

Chapter A: Backend API Reference

This appendix summarizes the core REST endpoints exposed by the Baligh Backend API. The API is designed using FastAPI and serves as the primary gateway for the React frontend to interact with the GEC, GED, and Tashkeel modules.

A.1. Core Endpoints

A.1.1. Analysis Router (`/api/analysis`)

- **Endpoint:** `POST /api/analysis`
- **Description:** Accepts a raw text payload from the editor, triggers the preprocessing pipeline, and runs the configured GED sub-detectors (Rules, Lexicon, ML). Returns a fused JSON list of error spans, categories, and explanations.
- **Response Structure:** List of `DetectionResponse` objects containing `start_index`, `length`, `category`, `tier`, and `explanation`.

A.1.2. Suggestions Router (`/api/suggestions`)

- **Endpoint:** `POST /api/suggestions`
- **Description:** Triggered when a user clicks on an identified error span. It forwards the context to the GEC Dictionary Engine to generate context-aware spelling and typographical alternatives based on Arramooz trie-lookups.
- **Response Structure:** List of `SuggestionResponse` objects ranked by probability.

A.1.3. Corrections Router (`/api/corrections`)

- **Endpoint:** `POST /api/corrections`
- **Description:** Triggers the GEC Ontology Engine. It receives the error span, queries the CAMeL feature mapper, generates SPARQL axioms, and returns the strictly enforced syntactic correction (e.g., forcing accusative case on a direct object).
- **Response Structure:** A `CorrectionResponse` object containing the guaranteed

valid phrase and an educational breakdown of the violated axiom.

A.1.4. Drafts Router (`/api/drafts`)

- **Endpoint:** `GET /api/drafts/{user_id}`
- **Description:** Fetches the user's saved writing sessions from the MongoDB backend using the Motor async driver.
- **Response Structure:** List of `DocumentDraft` objects containing `text`, `timestamp`, and `unresolved_errors`.

A.1.5. Tashkeel Router (`/api/tashkeel`)

- **Endpoint:** `POST /api/tashkeel`
- **Description:** Provides on-demand full diacritization (Tashkeel) for a given Arabic text payload, useful for pronunciation assistance and semantic disambiguation.